

Activity 10: Capstone: BudgetBuddy Release Evidence

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: A1, A2, A3, A4, A5

Estimated time: 105 minutes

Objective

Integrate the portable C++ core, Objective-C++ adapter and SwiftUI shell into a verified release evidence pack.

Deliverable

Working app, complete tests, architecture pack and deployment checklist

Tools

C++20, CMake, Xcode, SwiftUI, XCTest, documentation templates

Included files

- ``starter/capstone-brief.txt``
- ``starter/evidence-index.csv``
- ``starter/release-checklist.txt``
- ``solution/evidence-index.csv``
- ``solution/deployment-position.txt``

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run ``cd activities/10-capstone-release-evidence``, read ``starter/capstone-brief.txt``, and copy the checklist plus evidence index into ``workspace/``.
2. Create ``workspace/app``, ``workspace/tests``, ``workspace/docs`` and ``workspace/evidence``. Copy only the accepted outputs from Activities 1–9; exclude build caches, credentials, signing files and personal data.
3. Reconfirm the requirement baseline by comparing the Activity 1 matrix with the capstone brief. Record any change as an explicit decision before changing implementation.

4. Build and test the portable C++ core independently using ``cmake -S workspace/app/DomainCore -B workspace/app/DomainCore/build && cmake --build workspace/app/DomainCore/build && ctest --test-dir workspace/app/DomainCore/build --output-on-failure``, or the documented strict compiler fallback.
5. Build the Xcode scheme and run focused bridge tests. Record the scheme, destination and result; do not describe a Simulator build as an archive or App Store submission.
6. Run the SwiftUI golden journey: add an exact-cent expense, trigger and recover from an invalid entry, save, relaunch and verify stable identity and amount.
7. Run the accessibility critical path and persistence fault injection. Place raw results under ``workspace/evidence/`` and link them from the scorecard.
8. Reconcile C4 diagrams, ADRs and the public bridge contract against the final code. Resolve any dependency or ownership mismatch before marking documentation complete.
9. Complete ``workspace/evidence-index.csv`` with artifact version, requirement IDs, owner, evidence path and status. No PASS row may have an empty evidence field.
10. Review ``starter/release-checklist.txt``. Mark the archive item as planned unless a signed Release archive was actually created and validated; never infer App Store readiness from local tests.
11. Write ``workspace/docs/deployment-position.txt`` using the bounded pattern in the solution: distinguish local release-candidate evidence from signing, upload, processing and submission gates.
12. Run ``rg -n -i 'api[_]?key|secret|password|BEGIN .*PRIVATE KEY' workspace || true``. Investigate every match and remove real secrets before packaging.
13. Validate that every requirement has design, implementation, test and evidence links. Ask a reviewer to reproduce one domain check and one bridge/UI check from the index.
14. Create the final inventory with ``find workspace -type f | sort > workspace/evidence/file-inventory.txt``. Retain sources, fixtures, tests and evidence; remove only caches, derived data and temporary stores.

Verification

A clean evidence index proves requirements, implementation, interoperability, documentation and deployment readiness without claiming App Store submission.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

When a system test fails, localise the fault by rerunning domain then bridge tests before editing UI code.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Remove local build caches and secrets; retain source, fixtures, tests and evidence pack.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.